

Doc. no. P0467R1
Date: 2016-11-10
Project: Programming Language C++
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>
Audience: SG1, Library

Iterator Concerns for Parallel Algorithms

Revision History

Revision 0

Original version of the paper for the 2016 pre-Issaquah mailing.

Revision 1

Following approval from SG1 at Issaquah, normative wording added for the 2016 post-Issaquah mailing.

(It seems the document was not forwarded in time for the post-meeting mailing, and should make the pre-Kona mailing instead. This draft does not reflect changes to the draft since then - notably, all the parallel algorithm declarations have been duplicated in the body of the specification, so each edit should now be applied in two places.)

Table of Contents

- 0. [Revision History](#)
 - [Revision 0](#)
 - [Revision 1](#)
- 1. [Introduction](#)
- 2. [Problems to be addressed](#)
 - [The Standard Algorithms Are Underspecified](#)
 - [Input Iterators Invalidate Too Frequently](#)
 - [Copying an Input Range Requires Memory, and Copy Semantics](#)
 - [Writing to an Output Iterator Implies a Serialization Stage](#)
- 3. [Proposed Resolution](#)
 - [Which Algorithms Are Affected?](#)
- 4. [Alternatives Considered](#)
 - [Do Nothing](#)
 - [Add the Missing Parallel Overloads](#)
 - [Revise Requirements on all Algorithms](#)
 - [Blanket Wording Permitting Copies in Parallel Algorithms](#)
- 5. [Implementation Experience](#)
 - [Force Serial Execution](#)
 - [Copy Elements into Subranges](#)
 - [Ignore the Issue \(bug?\)](#)
- 6. [Formal Wording](#)
- 7. [References](#)
- 8. [Acknowledgements](#)

1. Introduction

The C++17 CD, [N4604](#), provides additional overloads for most of the algorithms in the `<algorithm>` and `<numeric>` headers, generally by repeating the declaration of the non-parallel form with a leading parallel-policy parameter. It is not clear that sufficient thought was given to the iterator category when providing such additions, and this paper looks into some of the resulting issues. As there are several reasonable directions to proceed from here, formal wording will not be provided until the first revision of this paper, following LEWG review.

2. Problems to be addressed

The main problem is that the parallel algorithms are underspecified. This paper claims that the poor specification naturally leads to those same algorithms being under-constrained, and suggests that different constraints may be appropriate for the serial and parallel forms of the same algorithm.

2.1 The Standard Algorithms Are Underspecified

To start, we must realize that many of the existing standard algorithms are under-constrained, at least when we want to extend their specification to cover the otherwise-unspecified parallel case.

Good examples of algorithms that have had their specification honed over several standard revisions are `adjacent_difference` and `partial_sum` in the `<numeric>` header. However, most algorithms are more like `copy`, and have poorly stated requirements that must be implicitly reverse-engineered from their *Effects:* clause.

2.2 Input Iterators Invalidate Too Frequently

The basic problem with an Input Iterator is that it does not provide the multi-pass guarantee. This makes it impossible for a parallel algorithm to work with different subsequences at the same time, as it is not possible to advance the iterator to refer to such subsequences without invalidating the iterators used by other threads.

2.3 Copying an Input Range Requires Memory, and Copy Semantics

One workaround for the lack of a multi-pass guarantee is to make copies of subsets of the input range, and dispatch work on those subsets. If this is the intended semantic, which is the understanding of the author, then it would be helpful to call this out in the parallel algorithms wording at the front of clause 25, so that future discussion in the Library Working Group dealing with parallel algorithms is informed of a deliberate design decision, calling out the runtime trade-offs and overheads accepted for the parallel forms. However, permission to create copies is not sufficient, as the algorithms do not currently require those elements to be CopyConstructible. For example, it is expected that the `move` algorithm work with `move_iterators`, possibly over sequences of `unique_ptr`s.

2.4 Writing to an Output Iterator Implies a Serialization Stage

Algorithms that write to Output Iterators do not currently require CopyConstructible elements; for example, an `ostream_iterator` requires just a reference in order to write the result to a stream.

Consider the following example:

```
template <typename T>
void WriteVector(std::vector<T> const & v, std::ostream & os) {
    copy(v.begin(), v.end(), std::ostream_iterator(os, ", "));
}
```

Despite the name of the algorithm being `copy`, we do not expect to make any actual copies executing with these iterators. For a clearer example, consider the following:

```
template <typename T>
void WriteVector(std::vector<std::unique_ptr<T>> const & v, std::ostream & os) {
    std::transform(v.begin(), v.end(),
        std::ostream_iterator(os, ", "),
        [](auto const &ptr){ return ptr.get(); });
}
```

In this case, we cannot copy elements from the source range, as `unique_ptr` does not have a usable copy constructor, but there is still an expectation that the algorithm should compile and run.

An output sequence supplied through a simple output iterator, such as the aforementioned `ostream_iterator`, does not offer the multi-pass guarantee; so again, output must be collected and output serially. It would be very surprising for the `copy` algorithms to write a permuted sequence, or for `merge` to produce a non-sorted output.

3. Proposed Resolution: Promote All Iterators to At Least Forward Iterators

The simplest short-term fix is to require that no iterator for a parallel algorithm has a category less capable than a Forward Iterator. This change guarantees that all iterators in parallel algorithms return a true reference, i.e., no copies are ever required, and the multi-pass guarantee allows the implementation to advance the destination (output) iterators without requiring additional storage to queue results for serialization.

This solution is recommended as it preserves the ability to weaken the constraints on iterators in a future library, once the implementation requirements are clear. It also grants time to the Library Working Group to properly document the constraints on the existing non-parallel algorithms.

3.1 Which Algorithms Are Affected?

Roughly 40 parallel algorithms use either input or output iterators in their interface, most of which use both.

Parallel Algorithms with Input Iterators and an Implicit Specification

The following algorithms all consume input ranges, may write to a simple output iterator, and make no clear requirement that the element type be copyable. Their specification is entirely implicit, not appearing beyond their corresponding header synopsis.

```
all_of
any_of
none_of
find
find_if
find_if_not
find_first_of
count
count_if
mismatch
equal
copy_n
copy_if
transform
replace_copy
replace_copy_if
remove_copy
```

```
remove_copy_if
partial_sort_copy
merge
includes
set_union
set_intersection
set_difference
set_symmetric_difference
lexicographical_compare
reduce
transform_reduce
inner_product
exclusive_scan
inclusive_scan
transform_exclusive_scan
transform_inclusive_scan
```

Parallel Algorithms with Input Iterators and an Explicit Specification

The following algorithms all consume input ranges, may write to a simple output iterator, and do not make a clear requirement that the element type be copyable. They do provide a distinct specification for the parallel form having a subtly different interface, such as `for_each`, or slightly different requirements, such as `copy`.

```
for_each
for_each_n
copy
move
```

Parallel Algorithms Writing Through Output Iterators

The following algorithms not already mentioned as consuming an input sequence require a serial (potentially copying) stage to write through an output iterator.

```
fill_n
generate_n
reverse_copy
rotate_copy
```

Potentially Well-specified Algorithms

The following algorithms over input ranges have implicitly specified parallel overloads, but may have a strong enough serial specification that the requirements are sufficient to allow for some parallel execution. However, even in these cases, careful review will likely suggest a minimal tweaking for clear support.

```
unique_copy
partition_copy
partial_sum
adjacent_difference
```

4. Alternatives Solutions

Several other resolutions were considered preparing this paper, and should be considered alongside the recommended resolution above.

4.1 Do Nothing

There is an intent that it should be very simple to turn serial C++ code into parallel code by simply adding a parallel-policy argument to the call of an algorithm. This becomes more troublesome if the user has to consider the potential for different requirements on the serial and parallel forms.

In particular, the standard gives no mandate that even those algorithms called with iterators that reveal embarrassing potential for parallelism should actually distribute the work, not least because the standard does not exclude environments that support only a single thread of execution. From this perspective, the parallel execution policies are no more than a hint to the library, like the `inline` keyword is a hint to the compiler. They allow for the relaxation of certain constraints (such as the ODR in the case of `inline`), subject to further constraints (e.g., identical definitions) that will retain proper behavior.

In this case, the promise made, beyond being a hint to the library that concurrent execution is desired, is the guarantee that passed function objects and iterators do not introduce data races when executed on different (unsynchronized) threads. This guarantee is a constraint on the user of the algorithm, rather than on the library implementation.

The author rejects this direction, as making a false offer of parallelism where serial implementations are effectively mandated is even more misleading. It would be better for the caller to confront the issues with their data structures that inhibit effective parallelism. Also, to some extent this risk is already assumed in the several algorithms that already have a different interface or contract for the serial and parallel forms, such as `for_each` and `copy`.

A further concern with this approach is that it has not been applied consistently through the library, as there are several algorithms with no parallel overloads, which could easily be given parallel overloads on the assumption that all known implementations would be sequential, but open to QoI enhancement in the future.

4.1a Add the Missing Parallel Overloads

This is an extension of the *do nothing* solution, which addresses the concern that the abstraction of parallel policy as a hint does not work as long as

there are some algorithms lacking the parallel policy overloads. Therefore, the policy parameter would be added to the remaining algorithms called out as *not* supported in the Parallelism Extensions TS.

4.2 Revise Requirements on all Algorithms

This resolution would likely require more effort from the Library Working Group than is available within the ballot resolution period for C++17. It would involve first correctly specifying all of the existing serial algorithms with a level of detail that is missing today. Once those specifications are complete, the parallel form of those same algorithms can be reviewed, and specified with additional constraints where necessary, such as the ability to take copies in order to create sub-tasks.

This resolution is the preferred long-term goal of the paper author, but seems beyond the scope of what can be achieved in two standard meetings, while addressing all of the other outstanding ballot comments. One possible result of this approach might be weaker requirements and a mandate to avoid undue serialization in the event of algorithms being called with Forward Iterators (or better) rather than simple Input or Output Iterators. At the least, it is an opportunity for the standard to acknowledge the greater capabilities that would come from such iterators.

4.3 Blanket Wording Permitting Copies in Parallel Algorithms

A half-way house would be to add additional blanket wording to the parallel algorithms front-matter, that says that any algorithm with a parallel policy overload taking input or output iterators as parameters, requires that the element type for such iterators be `CopyConstructible`.

While this direction would the progress from the status-quo, it suffers from still being an awkward specification for readers of the standard, who must be aware that there are now requirements on algorithms documented in several other places than the algorithm itself is specified, especially for parallel forms that are not otherwise specified at all. This could be partially alleviated by applying a blanket edit to actually specify those algorithms below the serial form, adding the `CopyConstructible` constraints in each case. Such a solution would still suffer from interactions with the under-specification of the serial forms though.

A trickier problem is that Output Iterators do not actually have an element type, so it is not clear what `CopyConstructible` constraint should apply to.

5. Implementation Experience

[P0024r1](#) provide links to existing practice prior to standardization. How do these implementations handle the problems highlighted in this paper?

5.1 Force Serial Execution

The Microsoft implementation at CodePlex simply forces all calls with input iterators into the sequential policy overloads - I did not find special handling for output iterators, but could easily have missed it in the time I had for this review.

5.2 Copy Elements into Subranges

The Khronos Sycl Parallel STL appears to try to copy elements into subranges, and performs parallel execution with the sub-ranges. The additional requirements for copyable elements do not appear to be documented, and as far as I can tell, calling with an incompatible element type would lead to a failure instantiating the template, deep in the implementation details. Libraries using this approach are best place to provide insights into the missing requirements, if that is the preferred direction.

5.3 Ignore the Issue (bug?)

Other libraries seemed to simply ignore the issue, working with potentially invalidated input (or output) iterators and risking undefined behavior. i.e., they have bugs, and lend no insight for a likely solution.

I am not calling out which of the unreferenced libraries fall into the categories above, as I did not have time for a detailed review, but these three categories appear to cover the range of the behavior I encountered.

6. Formal Wording

Header <algorithm>

synopsis

```
#include <initializer_list>

namespace std {
    // 25.3, non-modifying sequence operations:
    template <class InputIterator, class Predicate>
        bool all_of(InputIterator first, InputIterator last, Predicate pred);
    template <class ExecutionPolicy, class InputForwardIterator, class Predicate>
        bool all_of(ExecutionPolicy&& exec, // see 25.2.5
                   InputForwardIterator first, InputForwardIterator last, Predicate pred);
    template <class InputIterator, class Predicate>
        bool any_of(InputIterator first, InputIterator last, Predicate pred);
    template <class ExecutionPolicy, class InputForwardIterator, class Predicate>
        bool any_of(ExecutionPolicy&& exec, // see 25.2.5
                   InputForwardIterator first, InputForwardIterator last, Predicate pred);
    template <class InputIterator, class Predicate>
        bool none_of(InputIterator first, InputIterator last, Predicate pred);
    template <class ExecutionPolicy, class InputForwardIterator, class Predicate>
        bool none_of(ExecutionPolicy&& exec, // see 25.2.5
                   InputForwardIterator first, InputForwardIterator last, Predicate pred);
```

```

template<class InputIterator, class Function>
    Function for_each(InputIterator first, InputIterator last, Function f);
template<class ExecutionPolicy, class InputForwardIterator, class Function>
    void for_each(ExecutionPolicy&& exec, // see 25.2.5
        InputForwardIterator first, InputForwardIterator last, Function f);
template<class InputIterator, class Size, class Function>
    InputIterator for_each_n(InputIterator first, Size n, Function f);
template<class ExecutionPolicy, class InputForwardIterator, class Size, class Function>
    InputForwardIterator for_each_n(ExecutionPolicy&& exec, // see 25.2.5
        InputForwardIterator first, Size n, Function f);

template<class InputIterator, class T>
    InputIterator find(InputIterator first, InputIterator last,
        const T& value);
template<class ExecutionPolicy, class InputForwardIterator, class T>
    InputForwardIterator find(ExecutionPolicy&& exec, // see 25.2.5
        InputForwardIterator first, InputForwardIterator last,
        const T& value);
template<class InputIterator, class Predicate>
    InputIterator find_if(InputIterator first, InputIterator last,
        Predicate pred);
template<class ExecutionPolicy, class InputForwardIterator, class Predicate>
    InputForwardIterator find_if(ExecutionPolicy&& exec, // see 25.2.5
        InputForwardIterator first, InputForwardIterator last,
        Predicate pred);
template<class InputIterator, class Predicate>
    InputIterator find_if_not(InputIterator first, InputIterator last,
        Predicate pred);
template<class ExecutionPolicy, class InputForwardIterator, class Predicate>
    InputForwardIterator find_if_not(ExecutionPolicy&& exec, // see 25.2.5
        InputForwardIterator first, InputForwardIterator last,
        Predicate pred);
template<class ForwardIterator1, class ForwardIterator2>
    ForwardIterator1
        find_end(ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2, ForwardIterator2 last2);
template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
    ForwardIterator1
        find_end(ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2, ForwardIterator2 last2,
            BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
    ForwardIterator1
        find_end(ExecutionPolicy&& exec, // see 25.2.5
            ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2, ForwardIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1,
    class ForwardIterator2, class BinaryPredicate>
    ForwardIterator1
        find_end(ExecutionPolicy&& exec, // see 25.2.5
            ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2, ForwardIterator2 last2,
            BinaryPredicate pred);

template<class InputIterator, class ForwardIterator>
    InputIterator
        find_first_of(InputIterator first1, InputIterator last1,
            ForwardIterator first2, ForwardIterator last2);
template<class InputIterator, class ForwardIterator, class BinaryPredicate>
    InputIterator
        find_first_of(InputIterator first1, InputIterator last1,
            ForwardIterator first2, ForwardIterator last2,
            BinaryPredicate pred);
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class ForwardIterator2>
    InputIteratorForwardIterator1 find_first_of(ExecutionPolicy&& exec, // see 25.2.5
        InputIteratorForwardIterator1 first1, InputIteratorForwardIterator1 last1,
        ForwardIterator2 first2, ForwardIterator2 last2);
template<class ExecutionPolicy, class InputIteratorForwardIterator1,
    class ForwardIterator2, class BinaryPredicate>
    InputIteratorForwardIterator1
        find_first_of(ExecutionPolicy&& exec, // see 25.2.5
            InputIteratorForwardIterator1 first1, InputIteratorForwardIterator1 last1,
            ForwardIterator2 first2, ForwardIterator2 last2,
            BinaryPredicate pred);

template<class ForwardIterator>
    ForwardIterator adjacent_find(ForwardIterator first,
        ForwardIterator last);
template<class ForwardIterator, class BinaryPredicate>
    ForwardIterator adjacent_find(ForwardIterator first,
        ForwardIterator last,
        BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator adjacent_find(ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator first,
        ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class BinaryPredicate>
    ForwardIterator adjacent_find(ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator first,
        ForwardIterator last,

```

[illegible]

```

template<class ForwardIterator1, class ForwardIterator2>
    bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                        ForwardIterator2 first2);
template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
    bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                        ForwardIterator2 first2, BinaryPredicate pred);
template<class ForwardIterator1, class ForwardIterator2>
    bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                        ForwardIterator2 first2, ForwardIterator2 last2);
template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
    bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                        ForwardIterator2 first2, ForwardIterator2 last2,
                        BinaryPredicate pred);

// 25.3.13, search:
template<class ForwardIterator1, class ForwardIterator2>
    ForwardIterator1 search(
        ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2, ForwardIterator2 last2);
template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
    ForwardIterator1 search(
        ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2, ForwardIterator2 last2,
        BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
    ForwardIterator1 search(
        ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2, ForwardIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryPredicate>
    ForwardIterator1 search(ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2, ForwardIterator2 last2,
        BinaryPredicate pred);
template<class ForwardIterator, class Size, class T>
    ForwardIterator search_n(ForwardIterator first, ForwardIterator last,
        Size count, const T& value);
template<class ForwardIterator, class Size, class T, class BinaryPredicate>
    ForwardIterator search_n(ForwardIterator first, ForwardIterator last,
        Size count, const T& value,
        BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Size, class T>
    ForwardIterator search_n(ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator first, ForwardIterator last,
        Size count, const T& value);
template<class ExecutionPolicy, class ForwardIterator, class Size, class T,
        class BinaryPredicate>
    ForwardIterator search_n(ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator first, ForwardIterator last,
        Size count, const T& value,
        BinaryPredicate pred);

template <class ForwardIterator, class Searcher>
    ForwardIterator search(ForwardIterator first, ForwardIterator last,
        const Searcher &searcher);

// 25.4, modifying sequence operations:
// 25.4.1, copy:
template<class InputIterator, class OutputIterator>
    OutputIterator copy(InputIterator first, InputIterator last,
        OutputIterator result);
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class OutputIteratorForwardIterator2>
    OutputIteratorForwardIterator2 copy(ExecutionPolicy&& exec, // see 25.2.5
        InputIteratorForwardIterator1 first, InputIteratorForwardIterator1 last,
        OutputIteratorForwardIterator2 result);
template<class InputIterator, class Size, class OutputIterator>
    OutputIterator copy_n(InputIterator first, Size n,
        OutputIterator result);
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class Size,
        class OutputIteratorForwardIterator2>
    OutputIteratorForwardIterator2 copy_n(ExecutionPolicy&& exec, // see 25.2.5
        InputIteratorForwardIterator1 first, Size n,
        OutputIteratorForwardIterator2 result);
template<class InputIterator, class OutputIterator, class Predicate>
    OutputIterator copy_if(InputIterator first, InputIterator last,
        OutputIterator result, Predicate pred);
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class OutputIteratorForwardIterator2,
        class Predicate>
    OutputIteratorForwardIterator2 copy_if(ExecutionPolicy&& exec, // see 25.2.5
        InputIteratorForwardIterator1 first, InputIteratorForwardIterator1 last,
        OutputIteratorForwardIterator2 result, Predicate pred);
template<class BidirectionalIterator1, class BidirectionalIterator2>
    BidirectionalIterator2 copy_backward(
        BidirectionalIterator1 first, BidirectionalIterator1 last,
        BidirectionalIterator2 result);

// 25.4.2, move:
template<class InputIterator, class OutputIterator>
    OutputIterator move(InputIterator first, InputIterator last,
        OutputIterator result);

```

```

template<class ExecutionPolicy, class InputIteratorForwardIterator1,
        class OutputIteratorForwardIterator2>
OutputIteratorForwardIterator2 move(ExecutionPolicy&& exec, // see 25.2.5
        InputIteratorForwardIterator1 first, InputIteratorForwardIterator1 last,
        OutputIteratorForwardIterator2 result);
template<class BidirectionalIterator1, class BidirectionalIterator2>
BidirectionalIterator2 move_backward(
        BidirectionalIterator1 first, BidirectionalIterator1 last,
        BidirectionalIterator2 result);

// 25.4.3, swap:
template<class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 swap_ranges(ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 swap_ranges(ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2);
template<class ForwardIterator1, class ForwardIterator2>
void iter_swap(ForwardIterator1 a, ForwardIterator2 b);

template<class InputIterator, class OutputIterator, class UnaryOperation>
OutputIterator transform(InputIterator first, InputIterator last,
        OutputIterator result, UnaryOperation op);
template<class InputIterator1, class InputIterator2, class OutputIterator,
        class BinaryOperation>
OutputIterator transform(InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, OutputIterator result,
        BinaryOperation binary_op);
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class OutputIteratorForwardIterator2,
        class UnaryOperation>
OutputIteratorForwardIterator2 transform(ExecutionPolicy&& exec, // see 25.2.5
        InputIteratorForwardIterator1 first, InputIteratorForwardIterator1 last,
        OutputIteratorForwardIterator2 result, UnaryOperation op);
template<class ExecutionPolicy, class InputForwardIterator1, class InputForwardIterator2,
        class OutputForwardIterator, class BinaryOperation>
OutputForwardIterator transform(ExecutionPolicy&& exec, // see 25.2.5
        InputForwardIterator1 first1, InputForwardIterator1 last1,
        InputForwardIterator2 first2, OutputForwardIterator result,
        BinaryOperation binary_op);

template<class ForwardIterator, class T>
void replace(ForwardIterator first, ForwardIterator last,
        const T& old_value, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator, class T>
void replace(ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator first, ForwardIterator last,
        const T& old_value, const T& new_value);
template<class ForwardIterator, class Predicate, class T>
void replace_if(ForwardIterator first, ForwardIterator last,
        Predicate pred, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator, class Predicate, class T>
void replace_if(ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator first, ForwardIterator last,
        Predicate pred, const T& new_value);
template<class InputIterator, class OutputIterator, class T>
OutputIterator replace_copy(InputIterator first, InputIterator last,
        OutputIterator result,
        const T& old_value, const T& new_value);
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class OutputIteratorForwardIterator2, class T>
OutputIteratorForwardIterator2 replace_copy(ExecutionPolicy&& exec, // see 25.2.5
        InputIteratorForwardIterator1 first, InputIteratorForwardIterator1 last,
        OutputIteratorForwardIterator2 result,
        const T& old_value, const T& new_value);
template<class InputIterator, class OutputIterator, class Predicate, class T>
OutputIterator replace_copy_if(InputIterator first, InputIterator last,
        OutputIterator result,
        Predicate pred, const T& new_value);
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class OutputIteratorForwardIterator2,
        class Predicate, class T>
OutputIteratorForwardIterator2 replace_copy_if(ExecutionPolicy&& exec, // see 25.2.5
        InputIteratorForwardIterator1 first, InputIteratorForwardIterator1 last,
        OutputIteratorForwardIterator2 result,
        Predicate pred, const T& new_value);

template<class ForwardIterator, class T>
void fill(ForwardIterator first, ForwardIterator last, const T& value);
template<class ExecutionPolicy, class ForwardIterator,
        class T>
void fill(ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator first, ForwardIterator last, const T& value);
template<class OutputIterator, class Size, class T>
OutputIterator fill_n(OutputIterator first, Size n, const T& value);
template<class ExecutionPolicy, class OutputForwardIterator,
        class Size, class T>
OutputForwardIterator fill_n(ExecutionPolicy&& exec, // see 25.2.5
        OutputForwardIterator first, Size n, const T& value);

template<class ForwardIterator, class Generator>
void generate(ForwardIterator first, ForwardIterator last,
        Generator gen);

```

```

template<class ExecutionPolicy, class ForwardIterator, class Generator>
    void generate(ExecutionPolicy&& exec, // see 25.2.5
                 ForwardIterator first, ForwardIterator last,
                 Generator gen);
template<class OutputIterator, class Size, class Generator>
    OutputIterator generate_n(OutputIterator first, Size n, Generator gen);
template<class ExecutionPolicy, class OutputForwardIterator, class Size, class Generator>
    OutputForwardIterator generate_n(ExecutionPolicy&& exec, // see 25.2.5
                                     OutputForwardIterator first, Size n, Generator gen);

template<class ForwardIterator, class T>
    ForwardIterator remove(ForwardIterator first, ForwardIterator last,
                          const T& value);
template<class ExecutionPolicy, class ForwardIterator, class T>
    ForwardIterator remove(ExecutionPolicy&& exec, // see 25.2.5
                          ForwardIterator first, ForwardIterator last,
                          const T& value);
template<class ForwardIterator, class Predicate>
    ForwardIterator remove_if(ForwardIterator first, ForwardIterator last,
                             Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    ForwardIterator remove_if(ExecutionPolicy&& exec, // see 25.2.5
                             ForwardIterator first, ForwardIterator last,
                             Predicate pred);
template<class InputIterator, class OutputIterator, class T>
    OutputIterator remove_copy(InputIterator first, InputIterator last,
                              OutputIterator result, const T& value);
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class OutputIteratorForwardIterator2,
        class T>
    OutputIteratorForwardIterator2 remove_copy(ExecutionPolicy&& exec, // see 25.2.5
                                                InputIteratorForwardIterator1 first, InputIteratorForwardIterator1 last,
                                                OutputIteratorForwardIterator2 result, const T& value);
template<class InputIterator, class OutputIterator, class Predicate>
    OutputIterator remove_copy_if(InputIterator first, InputIterator last,
                                  OutputIterator result, Predicate pred);
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class OutputIteratorForwardIterator2,
        class Predicate>
    OutputIteratorForwardIterator2 remove_copy_if(ExecutionPolicy&& exec, // see 25.2.5
                                                  InputIteratorForwardIterator1 first, InputIteratorForwardIterator1 last,
                                                  OutputIteratorForwardIterator2 result, Predicate pred);

template<class ForwardIterator>
    ForwardIterator unique(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class BinaryPredicate>
    ForwardIterator unique(ForwardIterator first, ForwardIterator last,
                          BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator unique(ExecutionPolicy&& exec, // see 25.2.5
                          ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class BinaryPredicate>
    ForwardIterator unique(ExecutionPolicy&& exec, // see 25.2.5
                          ForwardIterator first, ForwardIterator last,
                          BinaryPredicate pred);
template<class InputIterator, class OutputIterator>
    OutputIterator unique_copy(InputIterator first, InputIterator last,
                              OutputIterator result);
template<class InputIterator, class OutputIterator, class BinaryPredicate>
    OutputIterator unique_copy(InputIterator first, InputIterator last,
                              OutputIterator result, BinaryPredicate pred);
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class OutputIteratorForwardIterator2>
    OutputIteratorForwardIterator2 unique_copy(ExecutionPolicy&& exec, // see 25.2.5
                                                InputIteratorForwardIterator1 first, InputIteratorForwardIterator1 last,
                                                OutputIteratorForwardIterator2 result);
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class OutputIteratorForwardIterator2,
        class BinaryPredicate>
    OutputIteratorForwardIterator2 unique_copy(ExecutionPolicy&& exec, // see 25.2.5
                                                InputIteratorForwardIterator1 first, InputIteratorForwardIterator1 last,
                                                OutputIteratorForwardIterator2 result, BinaryPredicate pred);

template<class BidirectionalIterator>
    void reverse(BidirectionalIterator first, BidirectionalIterator last);
template<class ExecutionPolicy, class BidirectionalIterator>
    void reverse(ExecutionPolicy&& exec, // see 25.2.5
                BidirectionalIterator first, BidirectionalIterator last);
template<class BidirectionalIterator, class OutputIterator>
    OutputIterator reverse_copy(BidirectionalIterator first,
                               BidirectionalIterator last,
                               OutputIterator result);
template<class ExecutionPolicy, class BidirectionalIterator, class OutputForwardIterator>
    OutputForwardIterator reverse_copy(ExecutionPolicy&& exec, // see 25.2.5
                                       BidirectionalIterator first,
                                       BidirectionalIterator last,
                                       OutputForwardIterator result);

template<class ForwardIterator>
    ForwardIterator rotate(ForwardIterator first,
                          ForwardIterator middle,
                          ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator rotate(ExecutionPolicy&& exec, // see 25.2.5
                          ForwardIterator first,

```

```

        ForwardIterator middle,
        ForwardIterator last);
template<class ForwardIterator, class OutputIterator>
    OutputIterator rotate_copy(
        ForwardIterator first, ForwardIterator middle,
        ForwardIterator last, OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class OutputIteratorForwardIterator2>
    OutputIteratorForwardIterator2 rotate_copy(
        ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator1 first, ForwardIterator1 middle,
        ForwardIterator1 last, OutputIteratorForwardIterator2 result);

// 25.4.12, sample:
template<class PopulationIterator, class SampleIterator,
        class Distance, class UniformRandomBitGenerator>
    SampleIterator sample(PopulationIterator first, PopulationIterator last,
        SampleIterator out, Distance n,
        UniformRandomBitGenerator&& g);

// 25.4.13, shuffle:
template<class RandomAccessIterator, class UniformRandomBitGenerator>
    void shuffle(RandomAccessIterator first,
        RandomAccessIterator last,
        UniformRandomBitGenerator&& g);

// 25.4.14, partitions:
template<class InputIterator, class Predicate>
    bool is_partitioned(InputIterator first, InputIterator last, Predicate pred);
template<class ExecutionPolicy, class InputForwardIterator, class Predicate>
    bool is_partitioned(ExecutionPolicy&& exec, // see 25.2.5
        InputForwardIterator first, InputForwardIterator last, Predicate pred);

template<class ForwardIterator, class Predicate>
    ForwardIterator partition(ForwardIterator first,
        ForwardIterator last,
        Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    ForwardIterator partition(ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator first,
        ForwardIterator last,
        Predicate pred);
template<class BidirectionalIterator, class Predicate>
    BidirectionalIterator stable_partition(BidirectionalIterator first,
        BidirectionalIterator last,
        Predicate pred);
template<class ExecutionPolicy, class BidirectionalIterator, class Predicate>
    BidirectionalIterator stable_partition(ExecutionPolicy&& exec, // see 25.2.5
        BidirectionalIterator first,
        BidirectionalIterator last,
        Predicate pred);
template<class InputIterator, class OutputIterator1,
        class OutputIterator2, class Predicate>
    pair<OutputIterator1, OutputIterator2>
    partition_copy(InputIterator first, InputIterator last,
        OutputIterator1 out_true, OutputIterator2 out_false,
        Predicate pred);
template<class ExecutionPolicy, class InputForwardIterator, class OutputForwardIterator1,
        class OutputForwardIterator2, class Predicate>
    pair<OutputForwardIterator1, OutputForwardIterator2>
    partition_copy(ExecutionPolicy&& exec, // see 25.2.5
        InputForwardIterator first, InputForwardIterator last,
        OutputForwardIterator1 out_true, OutputForwardIterator2 out_false,
        Predicate pred);
template<class ForwardIterator, class Predicate>
    ForwardIterator partition_point(ForwardIterator first,
        ForwardIterator last,
        Predicate pred);

// 25.5, sorting and related operations:
// 25.5.1, sorting:
template<class RandomAccessIterator>
    void sort(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    void sort(RandomAccessIterator first, RandomAccessIterator last,
        Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
    void sort(ExecutionPolicy&& exec, // see 25.2.5
        RandomAccessIterator first, RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    void sort(ExecutionPolicy&& exec, // see 25.2.5
        RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
    void stable_sort(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    void stable_sort(RandomAccessIterator first, RandomAccessIterator last,
        Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
    void stable_sort(ExecutionPolicy&& exec, // see 25.2.5
        RandomAccessIterator first, RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>

```

```

void stable_sort(ExecutionPolicy&& exec, // see 25.2.5
                RandomAccessIterator first, RandomAccessIterator last,
                Compare comp);

template<class RandomAccessIterator>
void partial_sort(RandomAccessIterator first,
                  RandomAccessIterator middle,
                  RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
void partial_sort(RandomAccessIterator first,
                  RandomAccessIterator middle,
                  RandomAccessIterator last, Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
void partial_sort(ExecutionPolicy&& exec, // see 25.2.5
                  RandomAccessIterator first,
                  RandomAccessIterator middle,
                  RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void partial_sort(ExecutionPolicy&& exec, // see 25.2.5
                  RandomAccessIterator first,
                  RandomAccessIterator middle,
                  RandomAccessIterator last, Compare comp);
template<class InputIterator, class RandomAccessIterator>
RandomAccessIterator partial_sort_copy(
    InputIterator first, InputIterator last,
    RandomAccessIterator result_first,
    RandomAccessIterator result_last);
template<class InputIterator, class RandomAccessIterator, class Compare>
RandomAccessIterator partial_sort_copy(
    InputIterator first, InputIterator last,
    RandomAccessIterator result_first,
    RandomAccessIterator result_last,
    Compare comp);
template<class ExecutionPolicy, class InputForwardIterator, class RandomAccessIterator>
RandomAccessIterator partial_sort_copy(
    ExecutionPolicy&& exec, // see 25.2.5
    InputForwardIterator first, InputForwardIterator last,
    RandomAccessIterator result_first,
    RandomAccessIterator result_last);
template<class ExecutionPolicy, class InputForwardIterator, class RandomAccessIterator, class Compare>
RandomAccessIterator partial_sort_copy(
    ExecutionPolicy&& exec, // see 25.2.5
    InputForwardIterator first, InputForwardIterator last,
    RandomAccessIterator result_first,
    RandomAccessIterator result_last,
    Compare comp);
template<class ForwardIterator>
bool is_sorted(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
bool is_sorted(ForwardIterator first, ForwardIterator last,
               Compare comp);
template<class ExecutionPolicy, class ForwardIterator>
bool is_sorted(ExecutionPolicy&& exec, // see 25.2.5
               ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
bool is_sorted(ExecutionPolicy&& exec, // see 25.2.5
               ForwardIterator first, ForwardIterator last,
               Compare comp);
template<class ForwardIterator>
ForwardIterator is_sorted_until(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
ForwardIterator is_sorted_until(ForwardIterator first, ForwardIterator last,
                               Compare comp);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator is_sorted_until(ExecutionPolicy&& exec, // see 25.2.5
                               ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
ForwardIterator is_sorted_until(ExecutionPolicy&& exec, // see 25.2.5
                               ForwardIterator first, ForwardIterator last,
                               Compare comp);

template<class RandomAccessIterator>
void nth_element(RandomAccessIterator first, RandomAccessIterator nth,
                 RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
void nth_element(RandomAccessIterator first, RandomAccessIterator nth,
                 RandomAccessIterator last, Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
void nth_element(ExecutionPolicy&& exec, // see 25.2.5
                 RandomAccessIterator first, RandomAccessIterator nth,
                 RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void nth_element(ExecutionPolicy&& exec, // see 25.2.5
                 RandomAccessIterator first, RandomAccessIterator nth,
                 RandomAccessIterator last, Compare comp);

// 25.5.3, binary search:
template<class ForwardIterator, class T>
ForwardIterator lower_bound(ForwardIterator first, ForwardIterator last,
                           const T& value);
template<class ForwardIterator, class T, class Compare>

```

```

    ForwardIterator lower_bound(ForwardIterator first, ForwardIterator last,
                                const T& value, Compare comp);

template<class ForwardIterator, class T>
    ForwardIterator upper_bound(ForwardIterator first, ForwardIterator last,
                                const T& value);
template<class ForwardIterator, class T, class Compare>
    ForwardIterator upper_bound(ForwardIterator first, ForwardIterator last,
                                const T& value, Compare comp);

template<class ForwardIterator, class T>
    pair<ForwardIterator, ForwardIterator>
        equal_range(ForwardIterator first, ForwardIterator last,
                    const T& value);
template<class ForwardIterator, class T, class Compare>
    pair<ForwardIterator, ForwardIterator>
        equal_range(ForwardIterator first, ForwardIterator last,
                    const T& value, Compare comp);

template<class ForwardIterator, class T>
    bool binary_search(ForwardIterator first, ForwardIterator last,
                       const T& value);
template<class ForwardIterator, class T, class Compare>
    bool binary_search(ForwardIterator first, ForwardIterator last,
                       const T& value, Compare comp);

// 25.5.4, merge:
template<class InputIterator1, class InputIterator2, class OutputIterator>
    OutputIterator merge(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2, InputIterator2 last2,
                        OutputIterator result);
template<class InputIterator1, class InputIterator2, class OutputIterator,
        class Compare>
    OutputIterator merge(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2, InputIterator2 last2,
                        OutputIterator result, Compare comp);
template<class ExecutionPolicy, class InputForwardIterator1, class InputForwardIterator2,
        class OutputForwardIterator>
    OutputForwardIterator merge(ExecutionPolicy&& exec, // see 25.2.5
                               InputForwardIterator1 first1, InputForwardIterator1 last1,
                               InputForwardIterator2 first2, InputForwardIterator2 last2,
                               OutputForwardIterator result);
template<class ExecutionPolicy, class InputForwardIterator1, class InputForwardIterator2,
        class OutputForwardIterator, class Compare>
    OutputForwardIterator merge(ExecutionPolicy&& exec, // see 25.2.5
                               InputForwardIterator1 first1, InputForwardIterator1 last1,
                               InputForwardIterator2 first2, InputForwardIterator2 last2,
                               OutputForwardIterator result, Compare comp);

template<class BidirectionalIterator>
    void inplace_merge(BidirectionalIterator first,
                      BidirectionalIterator middle,
                      BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
    void inplace_merge(BidirectionalIterator first,
                      BidirectionalIterator middle,
                      BidirectionalIterator last, Compare comp);
template<class ExecutionPolicy, class BidirectionalIterator>
    void inplace_merge(ExecutionPolicy&& exec, // see 25.2.5
                      BidirectionalIterator first,
                      BidirectionalIterator middle,
                      BidirectionalIterator last);
template<class ExecutionPolicy, class BidirectionalIterator, class Compare>
    void inplace_merge(ExecutionPolicy&& exec, // see 25.2.5
                      BidirectionalIterator first,
                      BidirectionalIterator middle,
                      BidirectionalIterator last, Compare comp);

// 25.5.5, set operations:
template<class InputIterator1, class InputIterator2>
    bool includes(InputIterator1 first1, InputIterator1 last1,
                 InputIterator2 first2, InputIterator2 last2);
template<class InputIterator1, class InputIterator2, class Compare>
    bool includes(InputIterator1 first1, InputIterator1 last1,
                 InputIterator2 first2, InputIterator2 last2, Compare comp);
template<class ExecutionPolicy, class InputForwardIterator1, class InputForwardIterator2>
    bool includes(ExecutionPolicy&& exec, // see 25.2.5
                 InputForwardIterator1 first1, InputForwardIterator1 last1,
                 InputForwardIterator2 first2, InputForwardIterator2 last2);
template<class ExecutionPolicy, class InputForwardIterator1, class InputForwardIterator2, class Compare>
    bool includes(ExecutionPolicy&& exec, // see 25.2.5
                 InputForwardIterator1 first1, InputForwardIterator1 last1,
                 InputForwardIterator2 first2, InputForwardIterator2 last2, Compare comp);

template<class InputIterator1, class InputIterator2, class OutputIterator>
    OutputIterator set_union(InputIterator1 first1, InputIterator1 last1,
                           InputIterator2 first2, InputIterator2 last2,
                           OutputIterator result);
template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
    OutputIterator set_union(InputIterator1 first1, InputIterator1 last1,
                           InputIterator2 first2, InputIterator2 last2,

```

[illegible]

```

    void push_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    void push_heap(RandomAccessIterator first, RandomAccessIterator last,
                    Compare comp);

template<class RandomAccessIterator>
    void pop_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    void pop_heap(RandomAccessIterator first, RandomAccessIterator last,
                    Compare comp);

template<class RandomAccessIterator>
    void make_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    void make_heap(RandomAccessIterator first, RandomAccessIterator last,
                    Compare comp);

template<class RandomAccessIterator>
    void sort_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    void sort_heap(RandomAccessIterator first, RandomAccessIterator last,
                    Compare comp);

template<class RandomAccessIterator>
    bool is_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    bool is_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
    bool is_heap(ExecutionPolicy&& exec, // see 25.2.5
                  RandomAccessIterator first, RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    bool is_heap(ExecutionPolicy&& exec, // see 25.2.5
                  RandomAccessIterator first, RandomAccessIterator last, Compare comp);
template<class RandomAccessIterator>
    RandomAccessIterator is_heap_until(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    RandomAccessIterator is_heap_until(RandomAccessIterator first, RandomAccessIterator last,
                                       Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
    RandomAccessIterator is_heap_until(ExecutionPolicy&& exec, // see 25.2.5
                                       RandomAccessIterator first, RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    RandomAccessIterator is_heap_until(ExecutionPolicy&& exec, // see 25.2.5
                                       RandomAccessIterator first, RandomAccessIterator last,
                                       Compare comp);

// 25.5.7, minimum and maximum:
template<class T> constexpr const T& min(const T& a, const T& b);
template<class T, class Compare>
    constexpr const T& min(const T& a, const T& b, Compare comp);
template<class T>
    constexpr T min(initializer_list<T> t);
template<class T, class Compare>
    constexpr T min(initializer_list<T> t, Compare comp);

template<class T> constexpr const T& max(const T& a, const T& b);
template<class T, class Compare>
    constexpr const T& max(const T& a, const T& b, Compare comp);
template<class T>
    constexpr T max(initializer_list<T> t);
template<class T, class Compare>
    constexpr T max(initializer_list<T> t, Compare comp);

template<class T> constexpr pair<const T&, const T&> minmax(const T& a, const T& b);
template<class T, class Compare>
    constexpr pair<const T&, const T&> minmax(const T& a, const T& b, Compare comp);
template<class T>
    constexpr pair<T, T> minmax(initializer_list<T> t);
template<class T, class Compare>
    constexpr pair<T, T> minmax(initializer_list<T> t, Compare comp);

template<class ForwardIterator>
    constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
    constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last,
                                         Compare comp);
template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator min_element(ExecutionPolicy&& exec, // see 25.2.5
                               ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
    ForwardIterator min_element(ExecutionPolicy&& exec, // see 25.2.5
                               ForwardIterator first, ForwardIterator last,
                               Compare comp);

template<class ForwardIterator>
    constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
    constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last,
                                         Compare comp);
template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator max_element(ExecutionPolicy&& exec, // see 25.2.5

```

```

        ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
    ForwardIterator max_element(ExecutionPolicy&& exec, // see 25.2.5
        ForwardIterator first, ForwardIterator last,
        Compare comp);

template<class ForwardIterator>
    constexpr pair<ForwardIterator, ForwardIterator>
        minmax_element(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
    constexpr pair<ForwardIterator, ForwardIterator>
        minmax_element(ForwardIterator first, ForwardIterator last, Compare comp);
template<class ExecutionPolicy, class ForwardIterator>
    pair<ForwardIterator, ForwardIterator>
        minmax_element(ExecutionPolicy&& exec, // see 25.2.5
            ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
    pair<ForwardIterator, ForwardIterator>
        minmax_element(ExecutionPolicy&& exec, // see 25.2.5
            ForwardIterator first, ForwardIterator last, Compare comp);

// 25.5.8, bounded value
template<class T>
    constexpr const T& clamp(const T& v, const T& lo, const T& hi);
template<class T, class Compare>
    constexpr const T& clamp(const T& v, const T& lo, const T& hi, Compare comp);

// 25.5.9, lexicographical comparison
template<class InputIterator1, class InputIterator2>
    bool lexicographical_compare(
        InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, InputIterator2 last2);
template<class InputIterator1, class InputIterator2, class Compare>
    bool lexicographical_compare(
        InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, InputIterator2 last2,
        Compare comp);
template<class ExecutionPolicy, class InputForwardIterator1, class InputForwardIterator2>
    bool lexicographical_compare(
        ExecutionPolicy&& exec, // see 25.2.5
        InputForwardIterator1 first1, InputForwardIterator1 last1,
        InputForwardIterator2 first2, InputForwardIterator2 last2);
template<class ExecutionPolicy, class InputForwardIterator1, class InputForwardIterator2, class Compare>
    bool lexicographical_compare(
        ExecutionPolicy&& exec, // see 25.2.5
        InputForwardIterator1 first1, InputForwardIterator1 last1,
        InputForwardIterator2 first2, InputForwardIterator2 last2,
        Compare comp);

// 25.5.10, permutations:
template<class BidirectionalIterator>
    bool next_permutation(BidirectionalIterator first,
        BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
    bool next_permutation(BidirectionalIterator first,
        BidirectionalIterator last, Compare comp);
template<class BidirectionalIterator>
    bool prev_permutation(BidirectionalIterator first,
        BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
    bool prev_permutation(BidirectionalIterator first,
        BidirectionalIterator last, Compare comp);
}

```

25.3.4 For each [alg.foreach]

```

template<class ExecutionPolicy, class InputForwardIterator, class Function>
    void for_each(ExecutionPolicy&& exec,
        InputForwardIterator first, InputForwardIterator last,
        Function f);

```

6. *Requires:* Function shall meet the requirements of CopyConstructible.
7. *Effects:* Applies *f* to the result of dereferencing every iterator in the range *[first, last)*. [*Note:* If the type of *first* satisfies the requirements of a mutable iterator, *f* may apply nonconstant functions through the dereferenced iterator. — *end note*]
8. *Complexity:* Applies *f* exactly *last - first* times.
9. *Remarks:* If *f* returns a result, the result is ignored.
10. *Notes:* Does not return a copy of its Function parameter, since parallelization may not permit efficient state accumulation.

```

template<class ExecutionPolicy, class InputForwardIterator, class Size, class Function>
    InputForwardIterator for_each_n(ExecutionPolicy&& exec, // see 25.2.5
        InputForwardIterator first, Size n, Function f);

```

16. *Requires:* Function shall meet the requirements of CopyConstructible.
17. *Requires:* *n* >= 0.
18. *Effects:* Applies *f* to the result of dereferencing every iterator in the range *[first, first + n)*. [*Note:* If the type of *first* satisfies the requirements of a mutable iterator, *f* may apply nonconstant functions through the dereferenced iterator. — *end note*]
19. *Returns:* *first + n*.

20. *Remarks:* If `f` returns a result, the result is ignored.

25.4.1 Copy [alg.copy]

```
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class OutputIteratorForwardIterator2>
OutputIteratorForwardIterator2 copy(ExecutionPolicy&& policy, InputIteratorForwardIterator1 first, InputIteratorForwardIterator1 last,
OutputIteratorForwardIterator2 result);
```

5. *Requires:* The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.
6. *Effects:* Copies elements in the range `[first, last)` into the range `[result, result + (last - first))`. For each non-negative integer `n` $< (last - first)$, performs $*(result + n) = *(first + n)$.
7. *Returns:* `result + (last - first)`.
8. *Complexity:* Exactly `last - first` assignments.

25.4.2 Move [alg.move]

```
template<class ExecutionPolicy, class InputIteratorForwardIterator1, class OutputIteratorForwardIterator2>
OutputIteratorForwardIterator2 move(ExecutionPolicy&& policy, InputIteratorForwardIterator1 first, InputIteratorForwardIterator1 last,
OutputIteratorForwardIterator2 result);
```

5. *Requires:* The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.
6. *Effects:* Moves elements in the range `[first, last)` into the range `[result, result + (last - first))`. For each non-negative integer `n` $< (last - first)$, performs $*(result + n) = std::move(*(first + n))$.
7. *Returns:* `result + (last - first)`.
8. *Complexity:* Exactly `last - first` assignments.

26.8 Generalized numeric operations [numeric.ops]

26.8.1 Header <numeric> synopsis [numeric.ops.overview]

```
namespace std {
    template <class InputIterator, class T>
        T accumulate(InputIterator first, InputIterator last, T init);
    template <class InputIterator, class T, class BinaryOperation>
        T accumulate(InputIterator first, InputIterator last, T init,
            BinaryOperation binary_op);

    template<class InputIterator>
        typename iterator_traits<InputIterator>::value_type
        reduce(InputIterator first, InputIterator last);
    template<class InputIterator, class T>
        T reduce(InputIterator first, InputIterator last, T init);
    template<class InputIterator, class T, class BinaryOperation>
        T reduce(InputIterator first, InputIterator last, T init,
            BinaryOperation binary_op);

    template<class ExecutionPolicy, class InputForwardIterator>
        typename iterator_traits<InputForwardIterator>::value_type
        reduce(ExecutionPolicy&& exec, // see 25.2.5
            InputForwardIterator first, InputForwardIterator last);
    template<class ExecutionPolicy, class InputForwardIterator, class T>
        T reduce(ExecutionPolicy&& exec, // see 25.2.5
            InputForwardIterator first, InputForwardIterator last, T init);
    template<class ExecutionPolicy, class InputForwardIterator, class T, class BinaryOperation>
        T reduce(ExecutionPolicy&& exec, // see 25.2.5
            InputForwardIterator first, InputForwardIterator last, T init,
            BinaryOperation binary_op);

    template<class InputIterator, class UnaryFunction, class T, class BinaryOperation>
        T transform_reduce(InputIterator first, InputIterator last,
            UnaryOperation unary_op, T init, BinaryOperation binary_op);
    template<class ExecutionPolicy, class InputForwardIterator,
        class UnaryFunction, class T, class BinaryOperation>
        T transform_reduce(ExecutionPolicy&& exec, // see 25.2.5
            InputForwardIterator first, InputForwardIterator last,
            UnaryOperation unary_op, T init, BinaryOperation binary_op);

    template <class InputIterator1, class InputIterator2, class T>
        T inner_product(InputIterator1 first1, InputIterator1 last1,
            InputIterator2 first2, T init);
    template <class InputIterator1, class InputIterator2, class T,
        class BinaryOperation1, class BinaryOperation2>
        T inner_product(InputIterator1 first1, InputIterator1 last1,
            InputIterator2 first2, T init,
            BinaryOperation1 binary_op1,
            BinaryOperation2 binary_op2);
    template <class ExecutionPolicy, class InputForwardIterator1, class InputForwardIterator2,
        class T>
        T inner_product(ExecutionPolicy&& exec, // see 25.2.5
            InputForwardIterator1 first1, InputForwardIterator1 last1,
            InputForwardIterator2 first2, T init);
    template <class ExecutionPolicy, class InputForwardIterator1, class InputForwardIterator2,
        class T, class BinaryOperation1, class BinaryOperation2>
        T inner_product(ExecutionPolicy&& exec, // see 25.2.5
            InputForwardIterator1 first1, InputForwardIterator1 last1,
            InputForwardIterator2 first2, T init,
            BinaryOperation1 binary_op1,
```

[illegible]

```

OutputIterator ForwardIterator2 transform_inclusive_scan(ExecutionPolicy&& exec, // see 25.2.5
    InputIterator ForwardIterator1 first, InputIterator ForwardIterator1 last,
    OutputIterator ForwardIterator2 result,
    UnaryOperation unary_op,
    BinaryOperation binary_op);

template<class ExecutionPolicy, class InputIterator ForwardIterator1, class OutputIterator ForwardIterator2,
    class UnaryOperation,
    class BinaryOperation, class T>
OutputIterator ForwardIterator2 transform_inclusive_scan(ExecutionPolicy&& exec, // see 25.2.5
    InputIterator ForwardIterator1 first, InputIterator ForwardIterator1 last,
    OutputIterator ForwardIterator2 result,
    UnaryOperation unary_op,
    BinaryOperation binary_op, T init);

template <class InputIterator, class OutputIterator>
OutputIterator adjacent_difference(InputIterator first,
    InputIterator last,
    OutputIterator result);

template <class InputIterator, class OutputIterator, class BinaryOperation>
OutputIterator adjacent_difference(InputIterator first,
    InputIterator last,
    OutputIterator result,
    BinaryOperation binary_op);

template <class ExecutionPolicy, class InputIterator ForwardIterator1, class OutputIterator ForwardIterator2>
OutputIterator ForwardIterator2 adjacent_difference(ExecutionPolicy&& exec, // see 25.2.5
    InputIterator ForwardIterator1 first,
    InputIterator ForwardIterator1 last,
    OutputIterator ForwardIterator2 result);

template <class ExecutionPolicy, class InputIterator ForwardIterator1, class OutputIterator ForwardIterator2,
    class BinaryOperation>
OutputIterator ForwardIterator2 adjacent_difference(ExecutionPolicy&& exec, // see 25.2.5
    InputIterator ForwardIterator1 first,
    InputIterator ForwardIterator1 last,
    OutputIterator ForwardIterator2 result,
    BinaryOperation binary_op);

template <class ForwardIterator, class T>
void iota(ForwardIterator first, ForwardIterator last, T value);

// 26.8.13, greatest common divisor
template <class M, class N>
constexpr common_type_t<M,N> gcd(M m, N n);

// 26.8.14, least common multiple
template <class M, class N>
constexpr common_type_t<M,N> lcm(M m, N n);
}

```

7. Acknowledgements

Thanks to David Sankel and Dietmar Kühl for early reviews of this paper, without necessarily endorsing it.

8. References

- [N4606](#) (Proposed) Working Draft for C++
- [N4507](#) C++ Extensions for Parallelism TS
- [P0024r1](#) The Parallelism TS Should be Standardized, Jared Hoberock
- <http://parallelstl.codeplex.com> Parallel STL at CodePlex
- <https://github.com/KhronosGroup/SyclParallelSTL> Khronos Group SyclParallelSTl